

DIA 8 — Resolvendo os Exercícios SQL de 9 a 17

Introdução

Nos exercícios anteriores, trabalhamos consultas básicas, filtros, ordenação, limitação de resultados, relacionamentos com `JOIN` e cálculos utilizando funções de agregação.

Agora vamos avançar para consultas um pouco mais completas.

Neste capítulo, veremos como:

- Contar registros com `COUNT` ;
- Calcular médias com `AVG` ;
- Encontrar valores máximos e mínimos;
- Somar valores calculados;
- Agrupar resultados com `GROUP BY` ;
- Filtrar agrupamentos com `HAVING` ;
- Remover registros duplicados com `DISTINCT` ;
- Utilizar subconsultas com `IN` ;
- Encontrar o maior resultado entre valores agrupados.

Mais importante do que memorizar os comandos é compreender o raciocínio por trás de cada solução.

A numeração considera a sequência iniciada no capítulo anterior. Como o exercício de contagem total de clientes já foi resolvido no Dia 7, começaremos pela consulta que conta produtos com preço superior a R\$ 20,00.

Exercício 9 — Mostrar a quantidade de produtos com preço maior que R\$ 20,00

O objetivo é descobrir quantos produtos possuem preço superior a R\$ 20,00.

```
SELECT
    COUNT(*) AS produtos_maiores_20
FROM produtos
WHERE preco > 20.00;
```

Entendendo a consulta

A cláusula `WHERE` é aplicada antes da função de agregação:

```
WHERE preco > 20.00
```

Primeiro, o banco seleciona apenas os produtos cujo preço seja maior que R\$ 20,00.

Depois, a função `COUNT(*)` conta quantas linhas permaneceram no resultado:

```
COUNT(*)
```

O alias:

```
AS produtos_maiores_20
```

atribui um nome mais claro à coluna retornada.

O resultado será semelhante a:

```
produtos_maiores_20
-----
3
```

Caso a regra fosse contar produtos com preço maior ou igual a R\$ 20,00, utilizaríamos:

```
WHERE preco >= 20.00
```

Exercício 10 — Mostrar o preço médio dos produtos

Neste exercício, precisamos calcular a média dos preços de todos os produtos cadastrados.

```
SELECT
    ROUND(AVG(preco), 2) AS preco_medio
FROM produtos;
```

Utilizando AVG

A função `AVG` calcula a média aritmética dos valores de uma coluna:

```
AVG(preco)
```

Internamente, o banco soma todos os preços e divide o resultado pela quantidade de valores considerados.

Por exemplo, considerando três produtos:

```
10,00  
20,00  
30,00
```

A média será:

```
(10 + 20 + 30) ÷ 3 = 20
```

Arredondando o resultado

Dependendo dos valores armazenados, a média pode possuir muitas casas decimais.

Por isso utilizamos a função `ROUND`:

```
ROUND(AVG(preco), 2)
```

O número `2` determina que o resultado deverá possuir duas casas decimais.

O alias:

```
AS preco_medio
```

torna a saída mais fácil de interpretar.

Exercício 11 — Mostrar o produto mais caro e o mais barato

Neste desafio, precisamos encontrar o maior e o menor preço cadastrados.

```
SELECT  
    MAX(preco) AS preco_maximo,  
    MIN(preco) AS preco_minimo  
FROM produtos;
```

Utilizando MAX

A função `MAX` retorna o maior valor encontrado:

```
MAX(preco)
```

Utilizando MIN

A função `MIN` retorna o menor valor:

```
MIN(preco)
```

Como as duas funções são agregações, o resultado será uma única linha:

```
preco_maximo | preco_minimo
-----|-----
50.00       | 1.89
```

É importante observar que essa consulta apresenta apenas os preços.

Ela não retorna os nomes dos produtos correspondentes.

Para descobrir o nome do produto mais caro, poderíamos utilizar:

```
SELECT
    nome_produto,
    preco
FROM produtos
ORDER BY preco DESC
LIMIT 1;
```

E para localizar o produto mais barato:

```
SELECT
    nome_produto,
    preco
FROM produtos
ORDER BY preco ASC
LIMIT 1;
```

Outra possibilidade seria utilizar subconsultas com `MAX` e `MIN`.

Exercício 12 — Mostrar o valor total das vendas

Agora precisamos calcular quanto foi vendido considerando todos os pedidos cadastrados.

O valor de cada item é calculado multiplicando:

preço do produto × quantidade comprada

A consulta é:

```
SELECT
    SUM(pr.preco * pe.qtde) AS valor_total_vendas
FROM pedidos pe
JOIN produtos pr
    ON pe.cod_produto = pr.cod_produto;
```

Por que precisamos do JOIN?

A quantidade comprada está na tabela `pedidos`:

`pe.qtde`

O preço está na tabela `produtos`:

`pr.preco`

Como as informações estão em tabelas diferentes, precisamos conectá-las:

```
JOIN produtos pr
    ON pe.cod_produto = pr.cod_produto
```

Calculando o valor de cada pedido

A expressão:

`pr.preco * pe.qtde`

calcula o valor correspondente a cada linha da tabela de pedidos.

Imagine um pedido com:

Preço: R\$ 45,00
Quantidade: 2

O valor calculado será:

45 × 2 = 90

Somando todas as vendas

A função `SUM` soma todos os valores calculados:

```
SUM(pr.preco * pe.qtde)
```

O resultado final representa o valor total de vendas registrado no banco.

Exercício 13 — Mostrar o valor total gasto por cada cliente

Neste exercício, precisamos calcular quanto cada cliente gastou.

```
SELECT
    c.nome_cliente,
    c.sobrenome_cliente,
    SUM(pr.preco * pe.qtde) AS total_gasto
FROM pedidos pe
JOIN clientes c
    ON pe.cod_cliente = c.cod_cliente
JOIN produtos pr
    ON pe.cod_produto = pr.cod_produto
GROUP BY
    c.nome_cliente,
    c.sobrenome_cliente
ORDER BY total_gasto DESC;
```

Relacionando as três tabelas

A consulta utiliza informações de três tabelas:

- `clientes`, para recuperar o nome;
- `pedidos`, para obter a quantidade;
- `produtos`, para obter o preço.

Primeiro, conectamos pedidos e clientes:

```
JOIN clientes c
ON pe.cod_cliente = c.cod_cliente
```

Depois, conectamos pedidos e produtos:

```
JOIN produtos pr
ON pe.cod_produto = pr.cod_produto
```

Agrupando por cliente

A função `SUM` precisa calcular um total separado para cada cliente.

Para isso, utilizamos:

```
GROUP BY
  c.nome_cliente,
  c.sobrenome_cliente
```

O `GROUP BY` reúne todos os pedidos pertencentes ao mesmo cliente.

Depois, esta expressão:

```
SUM(pr.preco * pe.qtde)
```

calcula o total gasto dentro de cada grupo.

O resultado será semelhante a:

nome_cliente	sobrenome_cliente	total_gasto
Maria	Oliveira	920.00
João	Silva	108.00
Pedro	Santos	32.00

Ordenando pelo maior gasto

A consulta termina com:

```
ORDER BY total_gasto DESC
```

Como o alias `total_gasto` representa o valor calculado, podemos utilizá-lo na ordenação.

O `DESC` apresenta primeiro o cliente que gastou mais.

Exercício 14 — Mostrar apenas os clientes que gastaram mais de R\$ 100,00

Agora precisamos calcular o total gasto por cliente e apresentar apenas aqueles cujo valor ultrapassou R\$ 100,00.

```
SELECT
    c.nome_cliente,
    c.sobrenome_cliente,
    SUM(pr.preco * pe.qtde) AS total_gasto
FROM pedidos pe
JOIN clientes c
    ON pe.cod_cliente = c.cod_cliente
JOIN produtos pr
    ON pe.cod_produto = pr.cod_produto
GROUP BY
    c.nome_cliente,
    c.sobrenome_cliente
HAVING SUM(pr.preco * pe.qtde) > 100
ORDER BY total_gasto DESC;
```

Por que não utilizar WHERE?

Pode parecer natural escrever:

```
WHERE SUM(pr.preco * pe.qtde) > 100
```

Entretanto, isso geraria erro.

A cláusula `WHERE` é executada antes do agrupamento e, portanto, não pode filtrar diretamente o resultado de funções como `SUM`.

Para filtrar grupos, utilizamos `HAVING`.

Entendendo o HAVING

A condição:


```
HAVING SUM(pr.preco * pe.qtde) > 100
```

é aplicada depois que os pedidos foram agrupados e o total de cada cliente foi calculado.

A sequência lógica da consulta é:

1. Ler os registros das tabelas;
2. Realizar os relacionamentos;
3. Agrupar os pedidos por cliente;
4. Calcular o total gasto;
5. Manter apenas os grupos com total acima de R\$ 100,00;
6. Ordenar o resultado.

A diferença principal é:

```
WHERE → filtra linhas antes do agrupamento  
HAVING → filtra grupos depois do agrupamento
```

Exercício 15 — Mostrar todos os produtos que já foram vendidos

O objetivo é listar os produtos que aparecem na tabela de pedidos.

```
SELECT DISTINCT  
    pr.nome_produto  
FROM pedidos pe  
JOIN produtos pr  
    ON pe.cod_produto = pr.cod_produto;
```

Por que utilizar DISTINCT?

Um produto pode aparecer em vários pedidos.

Sem o `DISTINCT`, o resultado poderia apresentar:

```
Refrigerante  
Pizza  
Refrigerante  
Cerveja  
Pizza
```

O `DISTINCT` remove as repetições:

```
SELECT DISTINCT pr.nome_produto
```

O resultado passaria a ser:

```
Refrigerante  
Pizza  
Cerveja
```

Cada nome aparece apenas uma vez, independentemente da quantidade de pedidos associados ao produto.

DISTINCT considera todas as colunas selecionadas

É importante observar que o `DISTINCT` analisa a combinação de todas as colunas apresentadas no `SELECT`.

Por exemplo:

```
SELECT DISTINCT  
    pr.nome_produto,  
    pe.qtde  
FROM pedidos pe  
JOIN produtos pr  
    ON pe.cod_produto = pr.cod_produto;
```

Nesse caso, duas linhas serão consideradas iguais somente se o nome do produto e a quantidade forem idênticos.

Exercício 16 — Mostrar produtos que possuem pedidos usando subconsulta

Neste exercício, precisamos recuperar os produtos que aparecem na tabela `pedidos`, mas utilizando uma subconsulta.

```
SELECT *  
FROM produtos  
WHERE cod_produto IN (  
    SELECT cod_produto  
    FROM pedidos  
);
```

Entendendo a consulta interna

A subconsulta:

```
SELECT cod_produto  
FROM pedidos
```

recupera os códigos dos produtos que aparecem em algum pedido.

O resultado interno pode ser semelhante a:

```
1  
3  
2  
5  
4
```

Utilizando IN

A consulta externa seleciona os produtos cujo código esteja dentro do conjunto retornado:

```
WHERE cod_produto IN (...)
```

Conceitualmente, seria semelhante a escrever:

```
WHERE cod_produto IN (1, 3, 2, 5, 4)
```

A diferença é que os códigos são descobertos automaticamente pela subconsulta.

Essa solução retorna os mesmos produtos que poderiam ser encontrados com um `JOIN`, mas utiliza uma estratégia diferente.

Utilizando EXISTS como alternativa

Outra forma de resolver o mesmo problema seria utilizar `EXISTS`:

```
SELECT  
  pr.*  
FROM produtos pr  
WHERE EXISTS (  
  SELECT 1
```

```
FROM pedidos pe
WHERE pe.cod_produto = pr.cod_produto
);
```

O `EXISTS` verifica se existe pelo menos uma linha correspondente na tabela `pedidos`.

Para cada produto, o banco pergunta:

Existe algum pedido relacionado a este produto?

Se a resposta for verdadeira, o produto será retornado.

Exercício 17 — Mostrar o cliente que mais gastou

Agora precisamos encontrar o cliente que possui o maior total de compras.

Essa consulta pode ser resolvida de diferentes formas.

Solução 1 — Utilizando subconsulta com MAX

```
SELECT
    nome_cliente,
    sobrenome_cliente,
    total_gasto
FROM (
    SELECT
        c.cod_cliente,
        c.nome_cliente,
        c.sobrenome_cliente,
        SUM(pr.preco * pe.qtde) AS total_gasto
    FROM pedidos pe
    JOIN clientes c
        ON pe.cod_cliente = c.cod_cliente
    JOIN produtos pr
        ON pe.cod_produto = pr.cod_produto
    GROUP BY
        c.cod_cliente,
        c.nome_cliente,
        c.sobrenome_cliente
) AS sub
WHERE total_gasto = (
    SELECT MAX(total_gasto)
    FROM (
        SELECT
```

```

        pe.cod_cliente,
        SUM(pr.preco * pe.qtde) AS total_gasto
FROM pedidos pe
JOIN produtos pr
    ON pe.cod_produto = pr.cod_produto
GROUP BY pe.cod_cliente
) AS max_sub
);

```

Primeira etapa: calcular o total por cliente

A primeira subconsulta produz uma tabela temporária semelhante a:

nome_cliente	sobrenome_cliente	total_gasto
João	Silva	108.00
Maria	Oliveira	920.00
Pedro	Santos	32.00

Essa tabela recebe o alias:

```
AS sub
```

Segunda etapa: descobrir o maior total

A subconsulta mais interna calcula novamente os totais agrupados:

```

SELECT
    pe.cod_cliente,
    SUM(pr.preco * pe.qtde) AS total_gasto
FROM pedidos pe
JOIN produtos pr
    ON pe.cod_produto = pr.cod_produto
GROUP BY pe.cod_cliente

```

Depois, a função `MAX` encontra o maior desses valores:

```
SELECT MAX(total_gasto)
```

Terceira etapa: localizar o cliente correspondente

A consulta externa compara:

```
WHERE total_gasto = maior_total
```

Assim, retorna o cliente cujo total gasto é igual ao maior valor encontrado.

Uma vantagem dessa solução é que, caso dois clientes estejam empatados com o mesmo maior valor, ambos serão apresentados.

Solução 2 — Utilizando ORDER BY e LIMIT

Uma forma mais simples é ordenar os clientes do maior total para o menor e limitar o resultado a uma linha.

```
SELECT
    c.nome_cliente,
    c.sobrenome_cliente,
    SUM(pr.preco * pe.qtde) AS total_gasto
FROM pedidos pe
JOIN clientes c
    ON pe.cod_cliente = c.cod_cliente
JOIN produtos pr
    ON pe.cod_produto = pr.cod_produto
GROUP BY
    c.cod_cliente,
    c.nome_cliente,
    c.sobrenome_cliente
ORDER BY total_gasto DESC
LIMIT 1;
```

Entendendo a solução

Primeiro, calculamos o total gasto por cliente.

Depois, ordenamos do maior para o menor:

```
ORDER BY total_gasto DESC
```

Por fim, retornamos somente o primeiro registro:

```
LIMIT 1
```

Essa abordagem é mais curta, legível e costuma ser suficiente quando queremos apenas um cliente.

Qual solução utilizar?

As duas soluções estão corretas, mas possuem comportamentos diferentes.

A solução com:

```
ORDER BY total_gasto DESC  
LIMIT 1
```

retorna apenas uma linha.

Caso dois clientes possuam exatamente o mesmo maior total, apenas um deles será apresentado, e sem outro critério de ordenação o escolhido pode não ser previsível.

Já a solução com `MAX` pode retornar todos os clientes empatados no maior valor.

Portanto:

- Utilize `ORDER BY` com `LIMIT 1` quando quiser somente um resultado;
- Utilize a comparação com `MAX` quando quiser considerar possíveis empates.

Melhorando os agrupamentos com o código do cliente

Nas consultas que agrupam clientes, é recomendável incluir o identificador único no `GROUP BY`.

Em vez de agrupar somente por:

```
GROUP BY  
  c.nome_cliente,  
  c.sobrenome_cliente
```

podemos utilizar:

```
GROUP BY  
  c.cod_cliente,  
  c.nome_cliente,  
  c.sobrenome_cliente
```

Isso evita que dois clientes diferentes, mas com o mesmo nome e sobrenome, sejam tratados como se fossem uma única pessoa.

Imagine dois clientes chamados João Silva.

Se a consulta agrupar apenas pelo nome, os pedidos dos dois poderão ser somados juntos.

Ao incluir `cod_cliente`, garantimos que cada cadastro seja tratado de forma independente.

Resumo dos Conceitos Utilizados

Neste capítulo trabalhamos as principais funções de agregação:

COUNT
AVG
MAX
MIN
SUM

Também utilizamos agrupamentos:

GROUP BY

E aprendemos a filtrar resultados agregados com:

HAVING

Para remover duplicidades, utilizamos:

DISTINCT

Para consultar valores retornados por outra consulta, usamos:

IN

Por fim, combinamos subconsultas com:

MAX

e vimos uma alternativa utilizando:

ORDER BY
LIMIT

Conclusão

As resoluções deste capítulo representam uma evolução importante no aprendizado de SQL.

Agora não estamos apenas recuperando registros individuais. Estamos transformando dados em informações úteis para o negócio.

Com essas consultas, podemos responder perguntas como:

- Quantos produtos custam mais de R\$ 20,00?
- Qual é o preço médio do catálogo?
- Qual é o valor total vendido?
- Quanto cada cliente gastou?
- Quais clientes ultrapassaram determinado valor?
- Quais produtos já foram vendidos?
- Qual cliente mais comprou?

Essas perguntas são comuns em relatórios, dashboards, sistemas financeiros, ERPs, e-commerces e aplicações corporativas.

Dominar funções de agregação, agrupamentos e subconsultas permite transformar um banco de dados em uma verdadeira fonte de conhecimento para a tomada de decisões.